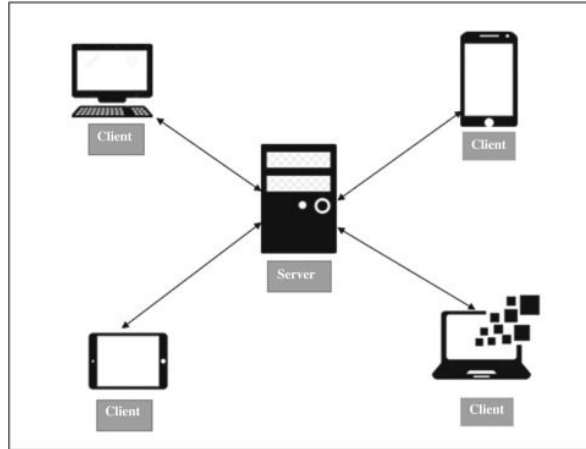




Database

A database is an organized collection of structured information or data stored electronically in a computer system



Database Management System (DBMS)

A Database Management System (DBMS) is a software application that allows users to create, maintain, and manage databases and their data. It acts as an interface between the database and users or applications, providing features to store, organize, retrieve, and update information efficiently and securely.

Relational Databases (RDBMS): Store data in structured tables with rows and columns
Examples - *MySQL* , *PostgreSQL*.

NoSQL Databases (Non-Relational): Store unstructured or flexible data without rigid tables. They scale easily across many servers. Example - *MongoDB*

What is a Relational Database (RDBMS)?

A relational database is a type of database that stores data in tables (also called relations), which are made up of rows and columns. Each table represents one type of entity (like students, products, or orders), and the tables can be linked or related to each other using keys (like primary key and foreign key).

What is a Non Relational Database or NOSQL?

A nonrelational database (also known as NoSQL database) is a type of database that does not store data in traditional tables with rows and columns. Instead, it stores data in flexible formats like documents, key value pairs, wide columns, or graphs, depending on the database type.

These databases are designed to handle large volumes of unstructured or semistructured data and provide scalability and speed, especially for web, big data, and real time applications.

What is SQL?

SQL (Structured Query Language) is the standard language used to communicate with relational databases. It is used to perform operations like `SELECT`, `INSERT`, `UPDATE`, `DELETE`, and more

What is MySQL?

MySQL is an open source relational database management system (RDBMS). It is used to store, manage, and retrieve data in a structured way using SQL (Structured Query Language).

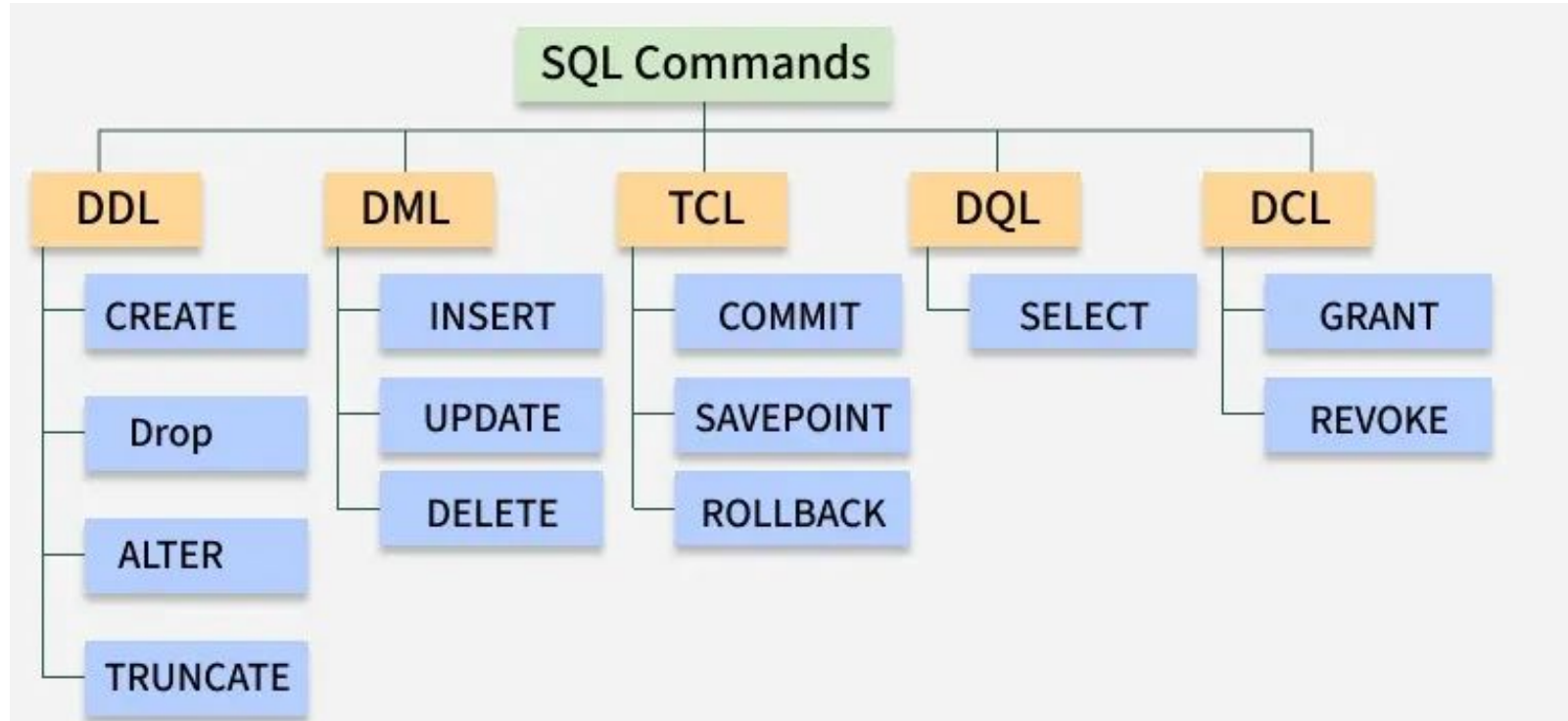
What are CRUD Operations?

CRUD operations refer to the four basic functions used to interact with a database: Create, Read, Update, and Delete.

These are essential for managing data in any application. The Create operation uses the `INSERT` command to add new data into a table, Read uses the `SELECT` command to retrieve existing data, Update uses the `UPDATE` command to modify current records, and Delete uses the `DELETE` command to remove data from the table.

Together, these operations form the backbone of how applications manage and manipulate data in a structured way.

Types of SQL Commands



Types of SQL Commands

1. **DDL** (Data Definition Language): These commands define and change the structure of a database. Examples are CREATE, ALTER, DROP, TRUNCATE.
2. **DML** (Data Manipulation Language): These commands are used to manage data inside tables. Examples are INSERT, UPDATE, DELETE.
3. **DQL** (Data Query Language): These commands are used to query or fetch data. The main command is SELECT.
4. **DCL** (Data Control Language): These commands control user access and permissions. Examples are GRANT, REVOKE.
5. **TCL** (Transaction Control Language): These commands manage transactions in a database. Examples are COMMIT, ROLLBACK, SAVEPOINT.

ACID Property

ACID is a set of four properties that ensure database transactions are processed reliably in MySQL.

- **Atomicity** means a transaction is treated as a single unit, so either all operations succeed or none are applied.
- **Consistency** ensures that a transaction moves the database from one valid state to another without violating rules or constraints.
- **Isolation** prevents multiple transactions running at the same time from interfering with each other, maintaining data accuracy.
- **Durability** guarantees that once a transaction is committed, its changes are permanently stored and will survive system failures, crashes, or power outages.

Together, these properties ensure data integrity and reliability.

Create and delete DataBase

CREATE DATABASE my_database; #This will create database

USE my_database; # To use a database

DROP DATABASE my_database; #Drop Database

Data Types in MySQL

Data types define the kind of data a column can hold, such as numbers, text, or dates. Choosing the right data type is crucial for efficient storage and query performance. They fall into three main categories:

- **Numeric**
- **String**
- **Date and Time.**

Numeric Data Types

Integer Types (exact value): For whole numbers. You can choose a type based on the size of the numbers you need to store to save space.

- **TINYINT:** Very small integers (-128 to 127).
- **SMALLINT:** Small integers (-32,768 to 32,767).
- **MEDIUMINT:** Medium Sized integers.(-8,388,608 to 8,388,607)
- **INT:** Standard integers(-2,147,483,648 to 2,147,483,647)(**most common**)
- **BIGINT:** Very large integers (+- 9,223,372,036,854,775,808)
- **DOUBLE:** Double Precision floating point number.(2.987,3.889)

String Data Types

- **CHAR(size):** A fixed length string. It always uses the same amount of storage space, regardless of the actual data length.
- **VARCHAR(size):** A variable length string. It only uses the space needed to store the data, plus a small overhead. It's the most common choice for general purpose strings like names and addresses.

Date and Time Data Types

- **DATE:** Stores a date in the format 'YYYY-MM-DD'.
- **TIME:** Stores a time in the format 'HH:MM:SS'.
- **DATETIME:** Stores both date and time in the format 'YYYY-MM-DD HH:MM:SS'.

Describe

Describe is used to find the structure of the table

```
DESCRIBE TABLE table_name;
```

Create, Alter and Drop a Table

How to Create a Table

You use the CREATE TABLE statement to define a new table.

```
⇒ CREATE TABLE employees (  
    employee_id INT ,  
    first_name VARCHAR(50),  
    last_name VARCHAR(50),  
    hire_date DATE  
);
```

How to use Comments

Use `or #` at the starting

```
-- This is a comment
```

```
# This is also a comment
```

How to Drop a Table?

```
DROP TABLE table_name;
```

Practice Questions

1. Create a table of sales with these columns Order_id, product, weight, qty, sales?
2. Create a table for library management system where there will be these table ?
 - Create a database Library
 - Create books_info table with book_id, name, pages, writer, year
 - Create stockbook_id, total_books, issued, remaining

How to make changes in table?

ALTER Command

The **ALTER** command in MySQL is a SQL statement used to modify the structure of an existing database table. It can be used for various tasks, such as adding, deleting, or modifying columns; adding or dropping constraints; and renaming the table.

Add a Column

You can use ALTER TABLE to add a new column to a table.

```
ALTER TABLE table_name  
ADD new_column_name datatype;
```

Modify a Column

The MODIFY COLUMN or ALTER COLUMN clause lets you change a column's data type, size, or other attributes.

```
ALTER TABLE table_name  
MODIFY COLUMN column_name new_datatype;
```

Change Column Name

To change a columns name use CHANGE COLUMN

```
alter table employee  
change column name emp_name varchar(30);
```

Drop a Column

To remove an existing column from a table, you use the DROP COLUMN clause.

```
ALTER TABLE table_name  
DROP COLUMN column_name;
```

Rename a Table

The RENAME TO clause is used to change the name of an entire table.

```
ALTER TABLE old_table_name  
RENAME TO new_table_name;
```

Insert, Update, and Delete Data

How to insert data into table?

Specify both the column names and the values:

```
INSERT INTO table_name (column1, column2, column3, ...)
VALUES (value1, value2, value3, ...);
```

→ `insert into employee(name,salary)`
`values("Mohan",34000);`

Specify only the values:

```
INSERT INTO table_name
VALUES (value1, value2, value3, ...);
```

→ `insert into employee`
`values("ravi",34000,"2026-2-3");`

How to insert data into table?

Multiple entries at a same time:

```
INSERT INTO Employees (employee_id, first_name, last_name, hire_date)
VALUES
  (103, 'Mary', 'Poppins', '2022-11-01'),
  (104, 'David', 'Beckham', '2021-03-25'),
  (105, 'Jessica', 'Alba', '2023-09-10');
```

How to insert NULL values

To insert a NULL value just type NULL at the time of entering data

```
insert into employee  
values(null,10000,null);
```

Safe Updates Mode : ERROR

MySQL's Safe Updates Mode (SQL_SAFE_UPDATES) prevents you from running an UPDATE or DELETE query that doesn't include a WHERE clause using a key column.

```
SET SQL_SAFE_UPDATES = 0;
```

How to Update an record?

UPDATE command is used to update any record

```
UPDATE table_name  
SET column1 = value1, column2 = value2, ...  
WHERE condition;
```

Delete and Truncate

DELETE

This query deletes the single row where the employee_id is 101.

```
DELETE FROM employees  
WHERE employee_id = 101;
```

Important Delete command

Deleting data in MySQL involves using the DELETE statement. It's a fundamental operation for removing one or more rows from a table. The primary way to control which rows get deleted is with a WHERE clause. Without a WHERE clause, the DELETE statement will remove all rows from the table.

Deleting All Rows

If you omit the WHERE clause, the DELETE statement will remove every row from the table.

```
DELETE FROM orders;
```

*(This is a powerful and destructive command. It's often better to use **TRUNCATE TABLE** for this purpose, as it's generally faster and resets the autoincrement counter.)*

TRUNCATE

TRUNCATE TABLE is a Data Definition Language (DDL) command in MySQL used to quickly remove all rows from a table. It's a highly efficient way to empty a table while keeping its structure intact.

```
TRUNCATE TABLE table_name;
```

TRUNCATE vs. DELETE

- **Speed:** TRUNCATE is significantly faster than DELETE because it deallocates the entire data space and doesn't log individual row deletions. DELETE is slower as it removes rows one by one and logs each removal.
- **WHERE Clause:** TRUNCATE cannot be used with a WHERE clause. It always deletes all rows. DELETE can use a WHERE clause to target specific rows.
- **Auto Increment:** TRUNCATE resets the AUTO_INCREMENT counter to its starting value (usually 1). DELETE does not reset the counter, so new rows will continue from the last value.
- **Rollback:** A TRUNCATE operation causes an implicit COMMIT, meaning it cannot be rolled back in MySQL.
- **Truncate** is a **DDL** command and **Delete** is a **DML** command.

Constraints

What are Constraints?

constraints are rules applied to columns in a table that restrict the data to maintain its accuracy and integrity. They prevent invalid or unwanted data from being inserted or updated in a database.

Types of Constraints

- **NOT NULL:** Ensures that a column cannot have a NULL (empty or missing) value.
- **UNIQUE:** Guarantees that all values in a column are distinct. No two records can have the same value in a column with this constraint.
- **CHECK:** Enforces a specific condition on the values in a column.
- **DEFAULT:** Specifies a default value for a column.

Not Null / Unique / Default constraint

```
create table test(  
    name varchar(30) not null,  
    phone int unique,  
    city varchar(40) default "indore"  
);
```

Check Constraint

```
create table vote(  
    name varchar(30),  
    age int,  
    constraint above_18 check (age>=18)  
);
```

```
show create table vote;
```



To check the constraint

```
insert into vote values("rohan",45); -- this will run  
insert into vote values("kunal",12); -- this will not
```

Error Code: 3819. Check constraint 'above_18' is violated.

Primary Key

A **primary key** is a column (or a set of columns) that uniquely identifies each row in a table. It's the most important identifier for a record. A table can only have one primary key, and it must meet two rules:

- **NOT NULL:** It cannot contain any NULL values.
- **UNIQUE:** Every value in the primary key column must be unique.

For example, in a customers table, a `customer_id` column would be a perfect primary key because each customer has a unique ID, and that ID is always present.

Primary Key

```
create table customer(  
    cid int primary key,  
    name varchar(30)  
);  
describe customer;
```

	Field	Type	Null	Key	Default	Extra
►	cid	int	NO	PRI	NULL	
	name	varchar(30)	YES		NULL	

Foreign Key

A **foreign key** is a column in one table that references the primary key of another table. It's the key that creates a link or relationship between two tables. The table with the foreign key is called the child table, and the table it references is the parent table. The foreign key ensures referential integrity, meaning you cannot have a value in the child table that doesn't exist in the parent table.

For example, consider an orders table. To link an order to a specific customer, you would include a `customer_id` column in the orders table.

Foreign Key

```
create table orders(  
    product varchar(30),  
    cid int,  
    sales int,  
    foreign key (cid) references customer(cid)  
);
```

AUTO_INCREMENT

```
create table customer
(  id int primary key auto_increment,
   name varchar(30)
);
```

Types Of Key Columns

1. Primary Key - Uniquely identifies each row
2. Foreign Key - Connects two tables
3. Unique Key - Prevents duplicate values
4. Composite Key - Combination of multiple columns
5. Candidate Key - Possible column that can become primary key
6. Alternate Key - Candidate key not chosen as primary
7. Super Key - Any column(s) that uniquely identify rows
8. Natural Key - Realworld meaningful value used as key
9. Surrogate Key - Artificial/generated key

Create A Copy Of A Table

```
create table new_emp as  
select * from employee;
```

← With data

```
create table new_emp like employee;
```

← Withoutdata

DQLData Query language

Data File for DQL

Please download this file.

https://github.com/deveshpatwa/Free_Datasets/blob/main/store.csv

SELECT

The SELECT command is the most fundamental and frequently used statement in SQL. It's a Data Query Language (DQL) command used to retrieve data from one or more tables in a database.

```
SELECT column1, column2, ...  
FROM table_name;
```

To get all the columns:

```
SELECT * FROM employees;
```

SELECT DISTINCT

The DISTINCT keyword is used in SQL to remove duplicate values from the result of a query. It ensures that only unique rows are returned.

```
SELECT DISTINCT country FROM customers;
```

```
select distinct *  
from store;
```



This will remove all the duplicate entry from the table

Aliases

Aliases provide temporary names for columns or tables, making queries easier to read and understand. They are defined using the AS keyword.

```
select name as 'Customer Name'  
from store;
```

OR

```
select name as Customer_Name  
from store;
```

WHERE

The WHERE clause is used to filter records. It specifies a condition that must be met for a row to be included in the result set. It's often used with

- comparison operators (=, !=, <, >, >=, <=)
- logical operators (AND, OR, NOT)

```
select category,sales
from store
where category = 'furniture'
and sales >= 1000;
```

```
select name,sales
from store
where name = 'Rick Hansen';
```

```
select name,sales
from store
where sales > 1000;
```

WHERE LIKE

The LIKE operator is always used with wildcard characters to define the search pattern. These wildcards allow you to match a variety of character combinations. There are two primary wildcards:

- **% (Percent sign):** Represents zero, one, or multiple characters.
- **_ (Underscore):** Represents a single character.

```
SELECT * FROM Customers WHERE CustomerName LIKE 'J%';
```

WHEREBETWEEN

The BETWEEN operator in MySQL is a logical operator used in the WHERE clause of a query to test whether an expression falls within a specified range of values. This range includes both the starting and ending values.

- Inclusiveness: The BETWEEN operator is inclusive
- Numeric values (e.g. between 10 and 50)
- Date/Time values (e.g. between '20250101' and '20250131')
- Text/String values (e.g. between 'M' and 'R', based on alphabetical order)

```
SELECT ProductID, ProductName, Price
FROM Products
WHERE Price BETWEEN 20.00 AND 50.00;
```

WHERE IS NULL

The IS NULL operator in MySQL (and SQL) is used in the WHERE clause to test for NULL values. You use it to find rows where a specific column has no value defined.

```
SELECT EmployeeID, FirstName, LastName  
FROM Employees  
WHERE DepartmentID IS NULL;
```

Where the value is not null:

```
SELECT EmployeeID, FirstName, LastName  
FROM Employees  
WHERE DepartmentID IS NOT NULL;
```

WHEREIN

The WHERE IN clause is used to filter rows based on whether a column's value matches any value in a given list.

```
SELECT *  
FROM customers  
WHERE country IN ('India', 'USA', 'UK');
```

ORDER BY

This clause is used to sort the result set in ascending (ASC) or descending (DESC) order based on one or more columns. The default order is ASC.

```
SELECT product_name, price FROM products ORDER BY price ASC;
```

For multiple columns:

```
select category, sub_category, sales  
from store  
order by category asc, sub_category asc, sales desc;
```

LIMIT

The LIMIT clause is used to restrict the number of rows returned by the query.

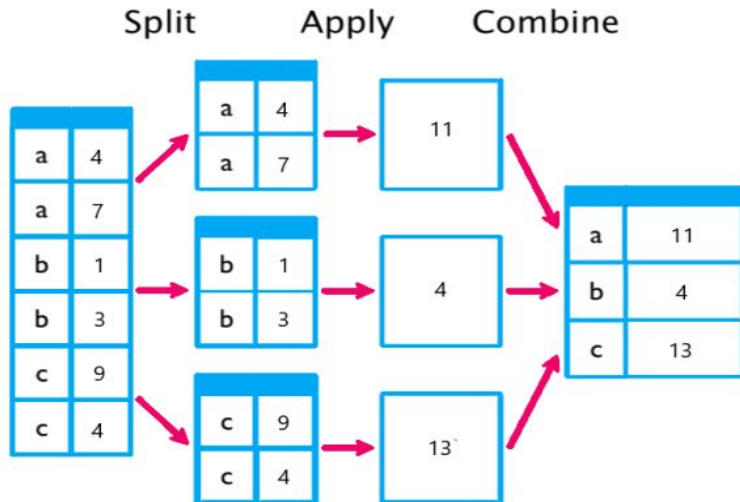
```
SELECT product_name, price FROM products  
  
ORDER BY price DESC LIMIT 10;
```

```
-- Find the customer name with the second most profitable transaction  
select name,profit  
from store  
order by profit desc  
limit 1 offset 1;
```

GROUP BY

The GROUP BY clause groups rows that have the same values into summary rows. It is most often used with aggregate functions like **COUNT()**, **MAX()**, **MIN()**, **SUM()**, and **AVG()** to perform calculations on each group.

```
SELECT department, COUNT(*) FROM employees GROUP BY department;
```



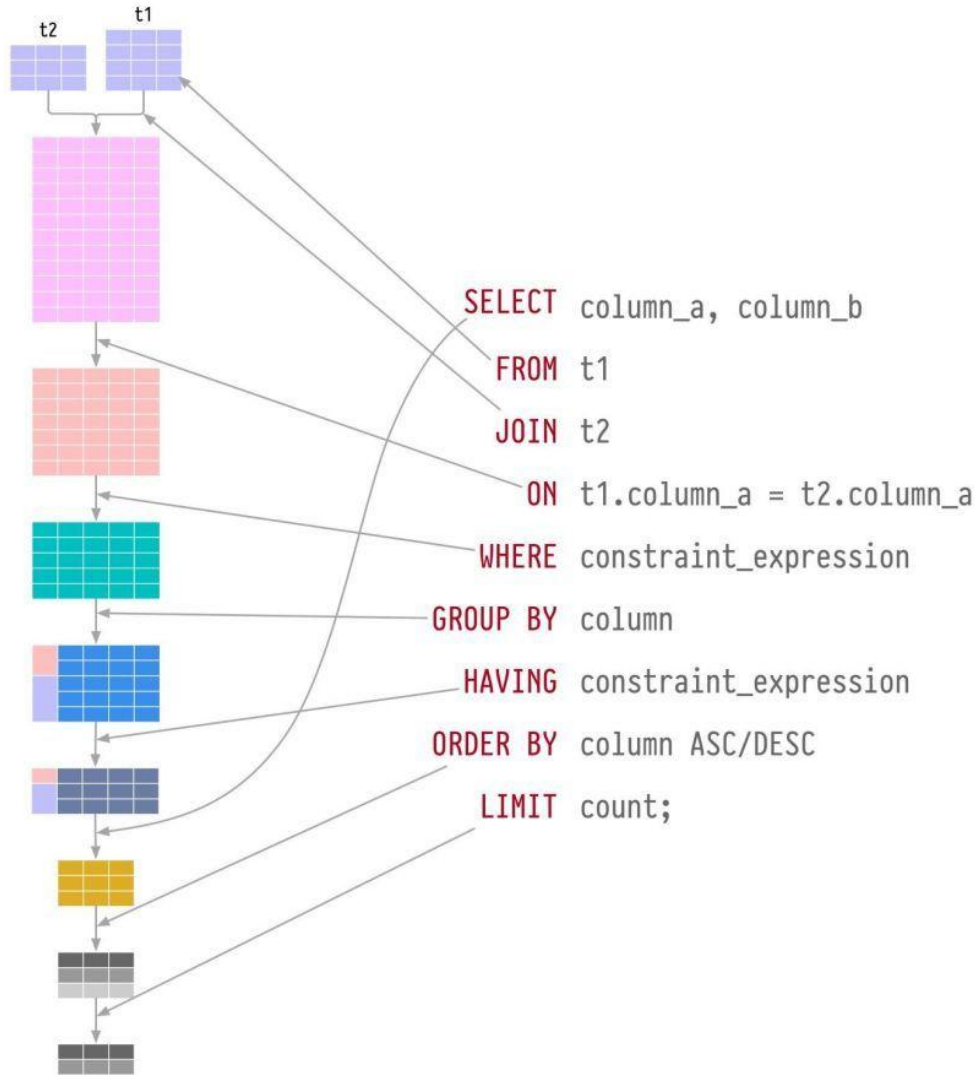
Practice Questions

1. Find subcategory wise total sales and total profit and sort them in an order where highest sales is on top ?
2. Find top five cities in east region with highest total sales ?
3. Find the customer who have generated the highest total profit and whose names start with 'A' ?
4. Find total number of orders where sales amount is less than 100 ?
5. Find the name of the customer with the second highest total quantity ?
6. Find the total number of order in each category ?
7. Find the subcategory wise total profit where we have made a loss ?
8. find low volume customer whose all time total purchase is less than one hundred dollars?

HAVING

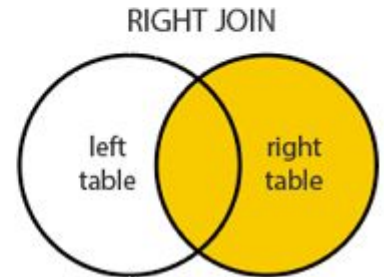
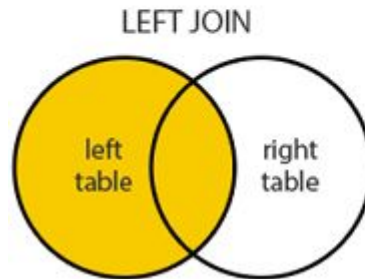
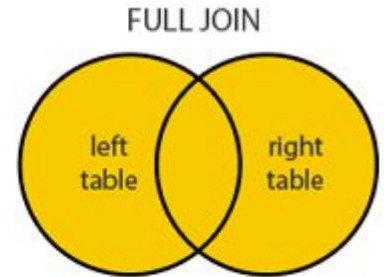
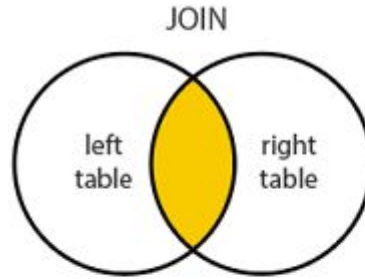
The HAVING clause is used to filter groups created by the GROUP BY clause. It is similar to WHERE but operates on aggregate results rather than individual rows.

```
SELECT department, COUNT(*)  
FROM employees  
GROUP BY department  
HAVING COUNT(*) > 5;
```



SQL Joins

SQL Joins are used to combine rows from two or more tables based on a related column between them. This is how you can retrieve data from multiple tables in a single query. The most common types of joins are INNER JOIN, LEFT JOIN, RIGHT JOIN, and FULL OUTER JOIN.



INNER JOIN

Imagine you have two tables: customers (with customer_id and name) and orders (with customer_id and amount). An INNER JOIN would return only the customer who are present in both tables (i.e., those who have a orders recorded).

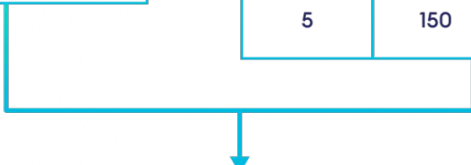
```
SELECT columns
FROM table1
INNER JOIN table2
ON table1.column_name = table2.column_name;
```

Table: Customers

customer_id	first_name
1	John
2	Robert
<u>3</u>	David
4	John
<u>5</u>	Betty

Table: Orders

order_id	amount	customer
1	200	10
2	500	<u>3</u>
3	300	6
4	800	<u>5</u>
5	150	8



customer_id	first_name	amount
3	David	500
5	Betty	800

LEFT JOIN

A LEFT JOIN returns all records from the left table and the matched records from the right table. If there's no match for a row in the right table, the query returns NULL for the columns from that table.

```
SELECT column_list
FROM table1
LEFT JOIN table2
ON table1.common_column = table2.common_column;
```

SQL LEFT JOIN

Table: Customers

customer_id	first_name
1	John
2	Robert
<u>3</u>	David
4	John
<u>5</u>	Betty

Table: Orders

order_id	amount	customer
1	200	10
2	500	<u>3</u>
3	300	6
4	800	<u>5</u>
5	150	8

customer_id	first_name	amount
1	John	
2	Robert	
3	David	500
4	John	
5	Betty	800

Full Join

- Its is not supported in MySQL
- but with the help of union and union all we can create a full join in mysql

UNION

```
select * from sales2019  
union  
select * from sales2020;
```

RIGHT JOIN

A RIGHT JOIN, also known as a RIGHT OUTER JOIN, returns all records from the right table and the matched records from the left table. If a record in the right table has no match in the left table, the columns from the left table will have NULL values.

```
SELECT column_list
FROM table1
RIGHT JOIN table2
ON table1.column_name = table2.column_name;
```

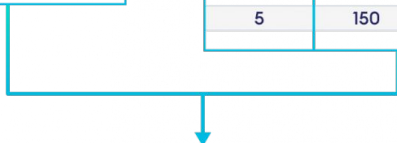
SQL RIGHT JOIN

Table: Customers

customer_id	first_name
1	John
2	Robert
3	David
4	John
5	Betty

Table: Orders

order_id	amount	customer
1	200	10
2	500	3
3	300	6
4	800	5
5	150	8



customer_id	first_name	amount
3	David	500
5	Betty	800
		200
		300
		150

Self Join

A self join is a query that joins a table to itself. This technique is used to compare rows within the same table as if you were joining two separate tables. To do this, you use table aliases, which are temporary names for the table in the query, allowing you to treat the single table as two distinct entities.

```
SELECT
    e.employee_name AS employee,
    m.employee_name AS manager
FROM
    employees AS e
INNER JOIN
    employees AS m ON e.manager_id = m.employee_id;
```

EmployeeID	FullName	Gender	ManagerID
1	Pranaya	Male	3
2	Priyanka	Female	1
3	Preety	Female	NULL
4	Anurag	Male	1
5	Sambit	Male	1
6	Rajesh	Male	3
7	Hina	Female	3

Subquery

A subquery, also known as a nested or inner query, is a query placed within another SQL statement. It's used to return data that will be used by the outer (main) query. Subqueries are always enclosed in parentheses and are executed first, providing a result set that the outer query then processes.

```
SELECT employee_name, salary
FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);
```

Questions for sub query

1. Find the number of records where the profit is below average ?
2. Find all the records where we have above average sales ?

WITH clause

The WITH clause in SQL, also known as a Common Table Expression (CTE), is a powerful feature that lets you define a temporary, named result set that you can reference within a SELECT, INSERT, UPDATE, or DELETE statement. It's essentially a temporary, virtual table that exists only for the duration of a single query.

```
with tt as
(select sub_category, round(sum(sales)) as total_sales
from store
group by sub_category)

select *
from tt
where total_sales >= 10000;
```

Index Columns

An index in MySQL is a specialized data structure that acts as a lookup table to significantly speed up data retrieval. Similar to a book's index, it allows the database engine to jump directly to specific data without scanning every single row in a table (a "full table scan").

```
create index indx_region on store(region);
```

```
show index from store;
```

```
drop index indx_city  
on store;
```

Functions in MySQL

TextFunctions

- **CONCAT(str1, str2, ...)** → joins strings.
- **LENGTH(str) / CHAR_LENGTH(str)** → returns string length.
- **LOWER(str) / UPPER(str)** → converts case.
- **TRIM(str)** → removes spaces.

Date and Time Functions

- `NOW()` → current date & time.
- `CURDATE()` / `CURRENT_DATE()` → current date.
- `CURTIME()` → current time.
- `DATE_ADD(date, INTERVAL n unit)` → add days/months/etc.
- `DATEDIFF(date1, date2)` → difference in days.
- `YEAR(date)`, `MONTH(date)`, `DAY(date)`, `WEEK(date)` → extract parts of a date.
- `STR_TO_DATE(str, format)` → convert string to date.

Mathematical Functions & Aggregate Functions

- **ROUND(number, decimals)** → rounds a number.
- **CEIL(number)** → smallest integer $\geq$ number.
- **FLOOR(number)** → largest integer $\leq$ number.
- **ABS(number)** → absolute value.
- **MOD(a, b)** → remainder of division.—Or you can use %
- **SUM(column)** → total.
- **AVG(column)** → average.
- **COUNT(column)** → number of rows.
- **MIN(column) / MAX(column)** → smallest/largest value.

sales	round(sales,1)	ceil(sales)	floor(sales)	profit	abs(profit)
44.02	44	45	44	11.4452	11.4452
2309.65	2309.6	2310	2309	762.1845	762.1845
1090.782	1090.8	1091	1090	-290.8752	290.8752
19.44	19.4	20	19	9.3312	9.3312
484.65	484.6	485	484	92.0835	92.0835
115.296	115.3	116	115	40.3536	40.3536
7.08	7.1	8	7	2.478	2.478
1.101	1.1	2	1	-2.5208	2.5208

Control Flow Functions

- IF(condition, value_if_true, value_if_false)
- CASE ... WHEN ... THEN ... ELSE ... END

CASE WHEN THEN ELSE END

The CASE statement in MySQL is a conditional expression that allows you to return a different value based on a series of conditions. It's similar to an IF THEN ELSE statement in programming

```
select sales,  
case  
    when sales<20 then "bad"  
    when sales<50 then "ok"  
    when sales<200 then "good"  
    else "very good"  
end as sales_category  
from store;
```

COALESCE

COALESCE is a function that takes a list of expressions as arguments and returns the first nonNULL value from that list. It evaluates the arguments from left to right.

```
SELECT
    user_name,
    COALESCE(phone_number, 'N/A') AS contact_info
FROM
    users;
```

Window Functions

Window functions are very useful in SQL. They allow you to perform calculations across a set of rows related to the current row without grouping the result.

```
SELECT column1,  
       window_function() OVER (  
         PARTITION BY column  
         ORDER BY column  
       )  
FROM table_name;
```

ROW_NUMBER()

ROW_NUMBER() gives a unique number to every row.

Even if two values are the same, the row number will still be different.

```
SELECT Customer,  
       Region,  
       Sales,  
       ROW_NUMBER() OVER(PARTITION BY Region ORDER BY Sales DESC) AS row_num  
FROM sales;
```

RANK()

If two rows have the same value, they get the same rank, and the next rank is skipped.

```
SELECT Student,  
       Marks,  
       RANK() OVER(ORDER BY Marks DESC) AS rank_num  
FROM students;
```

DENSE_RANK()

Same as RANK() but does NOT skip numbers.

```
SELECT Student,  
       Marks,  
       DENSE_RANK() OVER(ORDER BY Marks DESC) AS dense_rank  
FROM students;
```

LAG() – Previous Row Value

LAG() returns the value from the previous row.

LEAD() – Next Row Value

LEAD() returns the value from the next row.

Views

Views

A View in MySQL is a virtual table based on the resultset of a SQL query. It doesn't store data itself; instead, it stores the query that generates the data. When you query the view, MySQL runs the stored query and returns the result.

```
create view region_profile as
select
    region ,
    round(sum(sales)) as total_sales,
    round(sum(profit)) as total_profit
from store
group by region;

select * from region_profile;
```

Updatable vs. NonUpdatable

- **Updatable View:** A simple view (usually based on a single table) where you can use INSERT, UPDATE, and DELETE.
- **NonUpdatable View:** A complex view that uses things like sums, averages, or grouping (GROUP BY). You can only use SELECT on it.

Update a view

```
CREATE VIEW SimpleEmployeesView AS
SELECT EmployeeID, FirstName, LastName, DepartmentID
FROM Employees;

-- You can update a record using the view:
UPDATE SimpleEmployeesView
SET DepartmentID = 101
WHERE EmployeeID = 5;
-- This changes the DepartmentID in the underlying 'Employees' table.
```

Stored Procedures

Stored procedures in MySQL are prewritten SQL programs that are saved inside the database and can be executed whenever needed. Instead of writing the same SQL query again and again in your application, you store the logic once in the database and simply call it by name.

```
DELIMITER //  
  
CREATE PROCEDURE region_data(IN region_name varchar(30))  
BEGIN  
    SELECT name ,region, sales  
    FROM store  
    WHERE region = region_name;  
END //  
DELIMITER ;  
  
call regiondata("west");
```

Stored Procedures

- procedure is precompiled and stored in the database, execution can be faster compared to sending large queries again and again from the application.
- we use **DELIMITER** at the start because MySQL normally treats the semicolon ; as the end of a statement. > If MySQL is still using ; as the default delimiter, it will think the procedure ends at the first semicolon. That means it will not understand the full procedure definition.

Variables

In MySQL, there are mainly two types of variables:

First type is Userdefined variables (session variables).

User variable → Starts with @, works in session, no DECLARE needed.

Second type is Local variables (used inside stored procedures).

Local variable → Declared using DECLARE, works only inside procedure.

```
SET @age = 25;  
SELECT @age;
```

Functions

A function is a reusable block of SQL code that performs a specific task and always returns a single value. You create a function when you want to calculate something again and again, like tax calculation, age calculation, formatting text, or computing profit percentage.

There are two types of functions in MySQL.

- The first type is builtin functions, which MySQL already provides. SUM, MIN, AVG
- The second type is userdefined functions (UDFs), which you create yourself.

Functions

```
DELIMITER //
```

```
CREATE FUNCTION profit_percentage(sales DECIMAL(10,2), profit DECIMAL(10,2))  
RETURNS DECIMAL(5,2)  
DETERMINISTIC  
BEGIN  
    DECLARE result DECIMAL(5,2);  
    SET result = (profit / sales) * 100;  
    RETURN result;  
END //
```

```
DELIMITER ;
```

Trigger

A Trigger is a special type of stored program that runs automatically when a specific event happens on a table.

It executes automatically when:

- INSERT happens
- UPDATE happens
- DELETE happens

When Triggers are made

Triggers are based on timing and event.

- Based on Timing:

- BEFORE
- AFTER

BEFORE INSERT

AFTER INSERT

- Based on Event:

- INSERT
- UPDATE
- DELETE



BEFORE UPDATE

AFTER UPDATE

BEFORE DELETE

AFTER DELETE

```
DELIMITER //
```

```
CREATE TRIGGER log_deleted_employee
```

```
AFTER DELETE
```

```
ON employees
```

```
FOR EACH ROW
```

```
BEGIN
```

```
    INSERT INTO employees_backup(id, name, salary)
```

```
    VALUES(OLD.id, OLD.name, OLD.salary);
```

```
END //
```

```
DELIMITER ;
```